

SSU (ABNT NBR 14522) na TON-V2 — guia de referência

v1.2 · 21/09/2026 · protocolo, firmware, payload, testes e bancada · Apus Energia · Engenharia IoT



0 Resumo

O que chega	1 bloco por segundo: segundos restantes do intervalo, posto, quadrante e 2 contadores de pulsos (ativa, reativa) acumulados desde o início do intervalo.
Energia	Δ pulsos do mesmo registrador $\times K_e$ (kWh/pulso). Nunca comparar contadores de registradores diferentes.
Enlace	110 baud, 8N1, coletor aberto, sem delimitador — o bloco é fechado pelo silêncio.
Hardware	Borne X14 (X14-2 = SU+, X14-1 = GND) → IO48 · pull-up R47 680 Ω · TVS D4 · UART0, só RX.
Saída	Bucket por intervalo de demanda (15 min), JSON no formato do A-966, categoria Gateway no NexON.

1 Camada física

	Normal (3.4.1)	Estendido (3.4.2)
Velocidade	110 baud $\pm 3\%$	110 baud $\pm 3\%$
Caractere	start + 8 + 1-2 stop	start + 8 + 1 stop
Bloco / transmissão	8 octetos / ≈ 800 ms	9 octetos / ≈ 818 ms
Silêncio mínimo	≈ 200 ms	≈ 182 ms

Bit = 9,09 ms; caractere = ≈ 91 ms. Receber sempre em `SERIAL_8N1`. RX não invertido (idle alto pelo pull-up). Fechar o bloco em qualquer silêncio entre ~ 10 ms e 182 ms.



RX-timeout do ESP32 é em símbolos (caracteres), não bits. A 110 baud, 1 símbolo ≈ 91 ms. Usar `setRxTimeout(1)`; com 4 (≈ 364 ms) o bloco nunca fecha.

2 Formatos de bloco

Modo fixo de fábrica. O octeto 3 muda de significado entre os formatos → autodetecção: aceitar o formato só após **10 blocos seguidos** com tamanho + checksum válidos. Blocos da detecção não acumulam.

Normal — 8 octetos



Octeto	Bits	Conteúdo
1-2	1[7:0], 2[3:0]	Segundos restantes (12 bits, regressivo)
2	6-7	Pulsos capacitivos/indutivos computados para UFER/DMCR
3	0-3 · 4-5	Segmento horário (2 = fora de ponta nos vetores) · tarifa: 0 azul, 1 verde, 2 irrigantes, 3 outras
4-7	15 bits LE	Contadores ativa / reativa — máscara 0x7F no octeto alto; wrap em 32 768
8	—	LRC = ~(XOR dos octetos 1-7)

Estendido — 9 octetos (obrigatório com geração)



Octeto	Bits	Conteúdo
1-2	1[7:0], 2[3:0]	Segundos restantes (12 bits, regressivo)
2	4	Toggle a cada reposição de demanda (detectar mudança)
2	5	Toggle a cada fim de intervalo reativo (detectar mudança)
2	6-7	Posto reativo: 0 nenhum, 1 capacitivo, 2 indutivo, 3 ambos
3	0-1	Posto horário: 1 ponta, 2 fora de ponta, 3 quarto posto
3	2-3	Reservados — mascarar
3	4-5	Quadrante por tabela : 00 → Q1, 01 → Q4, 10 → Q2, 11 → Q3
3	7	Reativo sendo tarifado agora
4-7	16 bits LE	Contadores ativa / reativa: lo (hi << 8)
8-9	—	CRC-16/ARC: polinômio refletido 0xA001, init 0x0000, lo primeiro

3 Validação e decodificação

Checksum inválido → descartar o bloco inteiro e contar em `ssu_err`.

```
uint8_t lrc(const uint8_t* d, size_t n) {
    uint8_t v = 0;
    for (size_t i = 0; i < n; i++) v ^= d[i];
    return (uint8_t)~v;
}

uint16_t crc16(const uint8_t* d, size_t n) {
    uint16_t c = 0; // ≠ Modbus
    (0xFFFF)
    for (size_t i = 0; i < n; i++) {
        c ^= d[i];
        for (int b = 0; b < 8; b++)
            c = (c & 1) ? (c >> 1) ^ 0xA001 : c >> 1;
    }
    return c;
}
```

```
seg      = d[0] | ((d[1] & 0x0F) << 8);
bitRepos = d[1] & 0x10; // toggle
bitFimR  = d[1] & 0x20; // toggle
postoR   = (d[1] >> 6) & 3; // 0 · cap · ind ·
ambos
postoH   = d[2] & 3; // 1 ponta · 2 fora ·
3 quarto
q        = QUAD[(d[2] >> 4) & 3]; // {1,4,2,3}
tarifR   = d[2] & 0x80;
ativa    = d[3] | (d[4] << 8);
reativa  = d[5] | (d[6] << 8);
// normal: ativa = d[3] | ((d[4] & 0x7F) << 8)
//          seg. = d[2] & 0x0F; tarifa =
(d[2]>>4)&3
```

Exemplo: 3E 80 81 C0 2B 8E 12 55 B8

Octetos	Cálculo	Resultado
3E 80	$0 \times 3E + (0 \times 80 \& 0 \times 0F) \cdot 256$	62 s
80	$b7-6 = 10$	posto reativo indutivo
81	$b1-0 = 01 \cdot b5-4 = 00 \cdot b7 = 1$	ponta · Q1 · tarifando reativo
C0 2B / 8E 12	$0 \times 2BC0 / 0 \times 128E$	11 200 (REG1) / 4 750 (REG3)
55 B8	$CRC = 0 \times B855$	válido

4 Quadrantes e registradores



Reg	Conteúdo	JSON
REG1	ativa direta	phf
REG2	ativa reversa	phr
REG3	reativa Q1	qhfi
REG4	reativa Q2	qhri
REG5	reativa Q3	qhrc
REG6	reativa Q4	qhfc

```
regAtiva = (q == 1 || q == 4) ? 1 : 2;  
REG_R[5] = { 0, 3, 4, 5, 6 };  
regReativa = REG_R[q];
```

Não calcular quadrante como “bits + 1”

Acerta só Q1. 01 vira Q2 (consumo contado como injeção), 10 vira Q3 (reativo trocado), 11 vira Q4 (injeção contada como consumo). Sem erro visível — o checksum é válido.

5 Regras de acumulação

Situação	Sintoma	Tratamento
Troca de quadrante	Contador da ativa cai (11 210 → 41)	É outro registrador. Δ contra o último valor do REG2 (0 se o intervalo foi visto desde o início). Baseline só se o REG2 não foi visto desde o boot.
Wrap	65 500 → 40 no mesmo registrador	Δ modular: 76. Normal (15 bits): $\text{mod } 32\,768 - 32\,700 \rightarrow 40 = 108$.
Reinício do intervalo	Segundos sobem (0 → 899)	Publicar bucket, zerar acumuladores, base de todos = 0; o 1º bloco novo já conta.
Fechamento 3×	Blocos idênticos repetidos	Δ já dá 0; contar o evento de fechamento uma vez só.
Bits 4 e 5	Invertem a cada evento	Detectar mudança contra o último valor. Bit 5 ≈ a cada 4 intervalos (reativo 1 h).

```
const uint16_t MASK = normal ? 0x7FFF : 0xFFFF;
void processa(uint8_t reg, uint16_t atual) {          // reg = 1..6
    if (visto[reg]) acumula(reg, (atual - ultimo[reg]) & MASK);
    ultimo[reg] = atual; visto[reg] = true;
}
void reinicioIntervalo() {
    publicaBucket(); zeraAcumuladores();
    for (int r = 1; r <= 6; r++) { ultimo[r] = 0; visto[r] = true; }
}
```

6 Firmware na TON-V2

```
HardwareSerial ssu(0);                // ou Serial0 – não usar os dois
ssu.begin(110, SERIAL_8N1, 48, -1);    // RX = IO48, sem TX
ssu.setRxTimeout(1);                  // 1 símbolo ≈ 91 ms
ssu.onReceive(onBloco, true);          // callback só copia para fila FreeRTOS
```

Item	Comportamento
Parser	Roda no loop, lendo da fila. A recepção não depende do loop (SD, W5500, LoRa e Modbus bloqueiam por centenas de ms).
Bucket	acum[REG1..REG6] ; publicado quando os segundos reiniciam — alinhado ao intervalo faturado.
NVS	Acumuladores e NSU salvos a cada publicação e a cada 60 s.
Enlace degradado	30 blocos inválidos seguidos → sts = 0 e volta à autodetecção.
Alarme de formato	Bloco normal em instalação com geração → pedir saída estendida à concessionária.
Diagnóstico	/diagnostics: ssu_ok, ssu_err, ssu_fmt.
Gerador	Com “Medidor Concessionária (SSU)” no diagrama, o IO48 vira RX (sai do /inputs) e é gerado ssu.cpp . Lib: AupusNexOn/firmware-libs/ssu_nbr14522/ .
LoRa (TON4v2)	Bucket (≈ 120 B) pelo mesmo caminho dos devices Modbus; o gateway carimba a hora.

7 Payload e contas no NexON

```
{ "NSU": 32, "ver": "2.0.0-...",
  "data": { "cdo": "15", "phf": 0, "phr": 1532, "sts": 1, "qhfi": 0, "qhfc": 4, "qhri": 0, "qhrc": 0,
            "frame": "00 90 a2 fc 05 04 00 c6 18" },
  "time": "1789661700", "ke": 0.048, "fmt": 2, "q": 4, "seg": 898, "posto": 2, "ssu_err": 0 }
```

Campo	Significado
phf ... qhrc	Pulsos crus do bucket, 1:1 com REG1...REG6
ke	kWh/pulso do cadastro do medidor; sem ele o backend usa 0,048
fmt · q · seg · posto	Formato (1 normal, 2 estendido), quadrante, segundos e posto do último bloco
sts · ssu_err	Estado do enlace · blocos inválidos

Contas (bucket de 15 min): kWh = pulsos × Ke; kW médio = kWh ÷ 0,25. Ex.: 1 532 × 0,048 = 73,5 kWh → 294 kW exportados.

Exemplo inconsistente — virar caso de teste

O frame decodifica como Q2 (octeto 3 = A2), então o reativo 4 deveria ir para `qhri`, não `qhfc`. O `ssu.cpp` deve publicar `phr` = 1532, `qhri` = 4 para esse frame.

8 Testes automatizados

Vetor	Esperado
1C 82 92 D0 20 1C 0C 13	normal · 540 s · fora de ponta · verde · 8 400 / 3 100 · LRC 0x13
3E 80 81 C0 2B 8E 12 55 B8	Q1 · 62 s · ponta · indutivo · 11 200 (REG1) / 4 750 (REG3) · CRC 0xB855
82 43 32 29 00 0C 00 98 63	Q3 · 898 s · fora de ponta · capacitivo · 41 (REG2) / 12 (REG5) · CRC 0x6398
01 00 12 16 00 0a 00 f0	normal · 1 s · fora de ponta · verde · 22 / 10 · LRC 0xF0
00 90 a2 fc 05 04 00 c6 18	Q2 · 0 s · fora de ponta · indutivo · 1 532 (REG2) / 4 (REG4) · CRC 0x18C6

#	Caso (<code>test_ssu.cpp</code>)	Critério
1	Bloco normal no parser estendido	Rejeitado
2	Q1 → Q3	Sem pico; Δ REG2 = valor - base do REG2
3	Q1 → Q4	Ativa continua no REG1
4	3 fechamentos idênticos	1 evento
5	Bits 4/5 alternando	1 evento por transição
6	CRC/LRC com 1 bit errado	Rejeitado
7	Wrap 16 bits 65 500 → 40	Δ = 76
8	Wrap 15 bits 32 700 → 40	Δ = 108
9	Frame Q2 do payload	<code>phr</code> = 1532, <code>qhri</code> = 4
10	Enquadramento a 110 baud (simulador)	Todo callback com 8 ou 9 bytes, 1/s

9 Bancada: E750 + Conprove CE-6003

Ligação (tutorial Conprove E750 A2E3 — conferir na tampa do medidor)

Mala	E750	Obs.
V1, V2, V3 / comum	1, 2, 3 / 5	Medidor é autoalimentado: sem tensão, sem SSU
I1, I2, I3 → retornos	8, 9, 10 → 15, 14, 13	Retornos em ordem invertida
—	SU+ / SU-	X14-2 / X14-1 da TON-V2

Ângulos (tensões fixas em 0°, -120°, +120°; $\varphi = \angle V - \angle I$)

Alvo	φ	I1	I2	I3	SSU esperada
Q1	+25°	-25°	-145°	+95°	REG1 + REG3
Q4	-25°	+25°	-95°	+145°	REG1 + REG6
Q2	+155°	-155°	+85°	-35°	REG2 + REG4
Q3	-155°	+155°	+35°	-85°	REG2 + REG5

FP $0,92 = \pm 23,07^\circ$. No 1º ponto, conferir o quadrante no mostrador do E750; se vier invertido, trocar o sinal de φ em tudo.

Roteiro

- 1 Só tensão:** SSU transmitindo, formato travado, segundos decrescendo, contadores parados.
- 2 Ke de bancada:** Q1, $\varphi = 0$, 15 min (ex.: $66,4 \text{ V} \times 5 \text{ A} \times 3 \approx 996 \text{ W} \rightarrow 0,249 \text{ kWh}$). $Ke = \text{kWh injetados} \div \text{pulsos}$. Repetir com $\varphi = 90^\circ$ para o reativo.
- 3 Um quadrante por intervalo:** Q1, Q4, Q2, Q3 — energia só nos 2 registradores do quadrante.
- 4 Transições (Sequencer):** Q1 → Q4, Q1 → Q3, Q4 → Q3 no meio do intervalo, várias vezes — sem pico.
- 5 FP:** $\varphi = 23,0^\circ$ e $23,2^\circ$ — FP do NexON = $\cos \varphi$ injetado.
- 6 Longa duração:** cronometrar o bit 5, observar `ssu_err`, reboot no meio do intervalo (NVS).

10 Pendências

#	O quê	Onde	Critério de fechamento
1	Valor de <code>setRxTimeout</code> no código gerado	<code>_genSsuCpp</code>	Teste 10
2	Caso Q1 → Q3 do <code>test_ssu.cpp</code>	lib	$\Delta \text{REG2} = 41$ com intervalo visto
3	Wrap de 15 bits no bloco normal	lib	Teste 8
4	Mapeamento de registrador no payload	<code>ssu.cpp</code>	Teste 9
5	Uma só instância da UART0	código gerado	Sem <code>Serial0</code> + <code>HardwareSerial(0)</code>
6	Spec: <code>Serial2</code> , “bits+1 erra 2 de 4”, ângulos 155/205	spec	Spec alinhada a este guia
7	Convenção de ângulo do CTC; faixas da CE-6003	mala / CTC	Quadrante no mostrador = tabela
8	Ke de cadastro primário × Ke de bancada	parametrização E750	$Ke_{\text{prim}} = Ke_{\text{sec}} \times \text{RTC} \times \text{RTP}$, se aplicável
9	Contador 899 → 0 ou 900 → 1; intervalo reativo	bancada	Captura de uma virada; bit 5 cronometrado

Referências

ABNT NBR 14522:2008	§3.4.1 normal, §3.4.2 estendido, §3.4.2.6 registradores
Landis+Gyr E750 (A2E3)	Modo e constante da SU, intervalos de demanda e reativo
ESP32-S3 TRM / HardwareSerial.h	RX timeout em símbolos, <code>onReceive</code> , GPIO matrix
Conprove	Manual da CE-6003, CTC (Quick, Sequencer), tutorial E750 A2E3
Internos	Spec “Firmware de leitura da SSU” · <code>IOT-SSU-NBR14522-TON-V2-INTEGRACA0.md</code> · <code>firmware-libs/ssu_nbr14522/</code> · <code>mqtt.service.ts</code> > <code>salvarDadosGateway</code> · guia de hardware da TON-V2